

Geração de Relatórios de Desempenho no Framework IHP

Sistema de Gestão Acadêmica Funcional (SGA-Func) — Alunos • Turmas • Disciplinas • Notas • Frequência • Professores

UNIFOR

PROGRAMAÇÃO FUNCIONAL — 2026



1. Introdução

Sobre este Guia

Este guia explica como criar relatórios de desempenho no projeto SGA-Func utilizando o framework IHP. Os relatórios abrangem média de notas, frequência percentual, ranking de alunos e desempenho por turma e disciplina.

Conceitos de Programação Funcional Aplicados

O código utiliza intensamente os conceitos estudados na disciplina:

- **map** — transformação de listas
- **fold / reduce** — agregação de dados
- **Funções puras** — sem efeitos colaterais
- **Imutabilidade** — dados nunca alterados diretamente
- **Composição** — pipeline de operações encadeadas

Desempenho

Média de notas por aluno em cada turma

Frequência

Percentual de presença por aluno na turma

Ranking

Classificação geral de alunos ordenada por média

2. Pré-requisitos

1 Projeto IHP criado

Execute `ihp-new sga-func` para inicializar o projeto base do sistema.

2 Tabelas no Schema Designer

As tabelas `alunos`, `professores`, `disciplinas`, `turmas`, `notas` e `frequencias` devem estar criadas.

3 CRUD gerado

O CRUD para as entidades principais deve ter sido gerado via Code Generator do IHP.

4 Servidor rodando

Execute `./start` e confirme que o painel está disponível em `http://localhost:8001`.



3. Criar o Controller de Relatórios

No painel `http://localhost:8001`, acesse **Code Generator** → **New Controller** e crie o controller com o nome `Relatorios`. Em seguida, abra o arquivo `Web/Controller/Relatorios.hs` e implemente as actions descritas abaixo.

3.1 Cabeçalho do Módulo

```
module Web.Controller.Relatorios where

import Web.Controller.Prelude
import Web.View.Relatorios.Desempenho
import Web.View.Relatorios.Frequencia
import Web.View.Relatorios.Ranking
```

Actions Principais

DesempenhoAction { turmaId }

Calcula e exibe a média de notas por aluno em uma turma específica.

FrequenciaAction { turmaId }

Calcula e exibe o percentual de presença por aluno na turma.

RankingAction

Gera o ranking geral de alunos ordenado por média decrescente.

3.2 Funções Puras de Cálculo

Estas funções devem ficar no final do arquivo do Controller ou em um módulo auxiliar. Elas demonstram na prática os conceitos de **map**, **fold** e **imutabilidade** exigidos pela disciplina.

```
calcularDesempenho :: [Nota] -> [(Id Aluno, Double)]
calcularDesempenho notas =
  map \(alunoId, ns) -> (alunoId, media ns))
  $ Map.toList
  $ foldr (\n acc -> Map.insertWith (++) (get #alunoId n) [get #valor n] acc)
    Map.empty notas
  where media xs = sum xs / fromIntegral (length xs)

calcularFrequencia :: [Frequencia] -> [(Id Aluno, Double)]
calcularFrequencia freqs =
  map \(alunoId, fs) -> (alunoId, percentual fs))
  $ Map.toList
  $ foldr (\f acc -> Map.insertWith (++) (get #alunoId f) [get #presente f] acc)
    Map.empty freqs
  where percentual fs = (fromIntegral $ length $ filter id fs) / fromIntegral (length fs) * 100

calcularRanking :: [Nota] -> [(Id Aluno, Double)]
calcularRanking = sortBy (comparing (Down . snd)) . calcularDesempenho
```

❏ Nenhuma dessas funções altera dados existentes — elas apenas transformam listas de entrada em novas listas de saída, respeitando o princípio da imutabilidade.

3.2 Funções Puras de Cálculo

```
calcularDesempenho :: [Nota] -> [(Id Aluno, Double)]
calcularDesempenho notas =
  map (\(alunoId, ns) -> (alunoId, media ns))
    $ Map.toList
    $ foldr (\n acc -> Map.insertWith (++) (get #alunoId n) [get #valor n] acc)
      Map.empty notas
  where media xs = sum xs / fromIntegral (length xs)

calcularFrequencia :: [Frequencia] -> [(Id Aluno, Double)]
calcularFrequencia freqs = map (\(alunoId, fs) -> (alunoId, percentual fs))
  $ Map.toList
  $ foldr (\f acc -> Map.insertWith (++) (get #alunoId f) [get #presente f] acc)
    Map.empty freqs
  where percentual fs = (fromIntegral $ length $ filter id fs) / fromIntegral (length fs) * 100

calcularRanking :: [Nota] -> [(Id Aluno, Double)]
calcularRanking = sortBy (comparing (Down . snd)) . calcularDesempenho
```

- ❏ Nenhuma dessas funções altera dados existentes — elas apenas transformam listas de entrada em novas listas de saída, respeitando o princípio da imutabilidade.

4. Criar as Views

Crie os arquivos de view dentro da pasta `Web/View/Relatorios/`. Cada view recebe os dados já calculados pelo Controller e os renderiza em HTML utilizando a sintaxe **HSX** do IHP.

4.1 View de Desempenho

`Web/View/Relatorios/Desempenho.hs`

Recebe a turma, a média geral da turma e a lista de desempenho por aluno. Utiliza HSX para renderizar uma tabela HTML com o nome (ou ID) do aluno e sua respectiva média calculada.

4.2 View de Frequência

`Web/View/Relatorios/Frequencia.hs`

Exibe o percentual de presença de cada aluno na turma selecionada. A tabela mostra o aluno e o percentual formatado, permitindo identificar rapidamente alunos com baixa frequência.

4.3 View de Ranking

`Web/View/Relatorios/Ranking.hs`

Lista todos os alunos do sistema ordenados da maior para a menor média geral, consolidando notas de todas as turmas. Ideal para visão global do desempenho acadêmico.

5. Configurar as Rotas

No arquivo de rotas (`Web/Routes.hs` ou equivalente), adicione a linha abaixo para registrar automaticamente todas as rotas do controller de relatórios:

```
instance AutoRoute RelatoriosController
```

Por que AutoRoute?

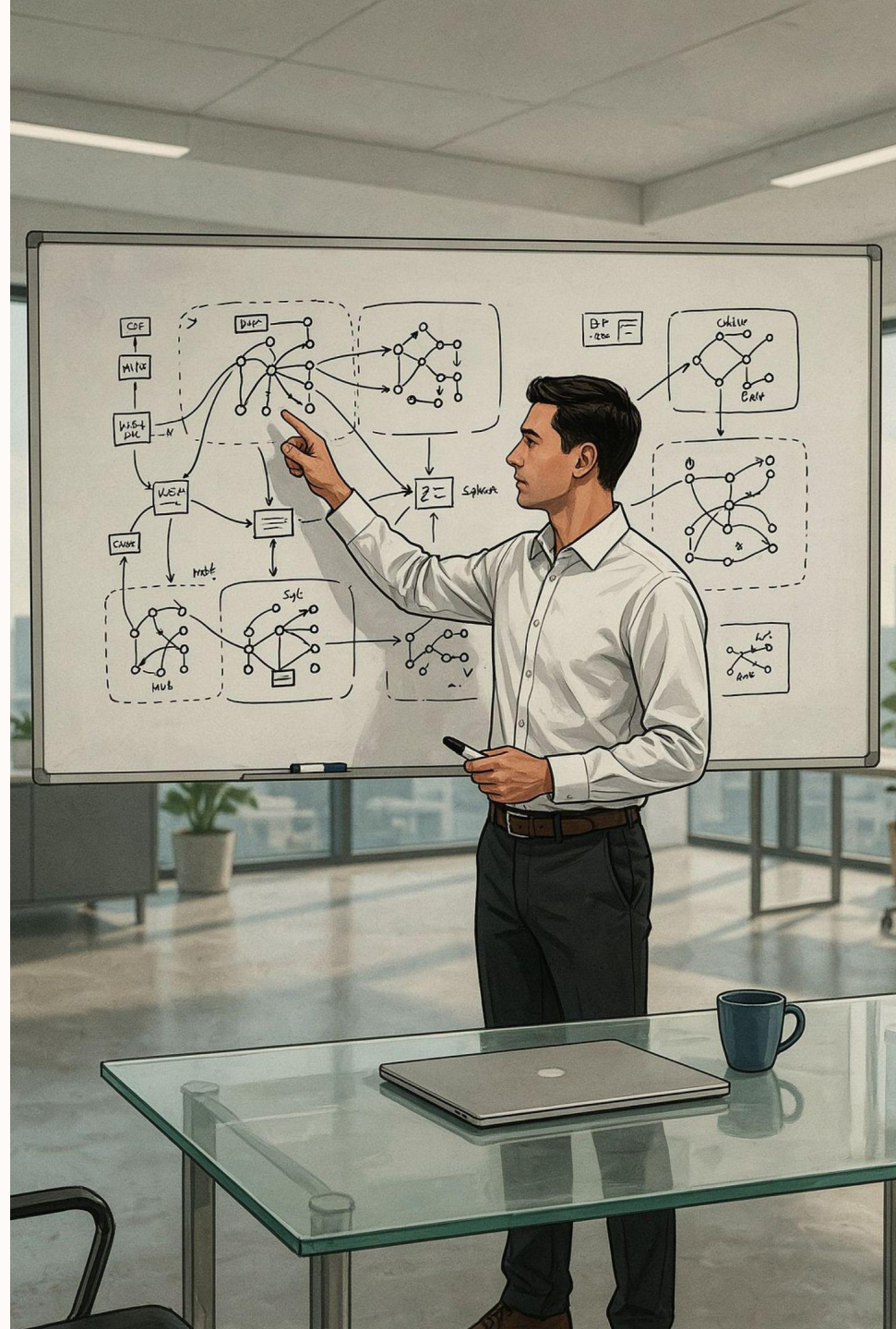
O AutoRoute do IHP gera automaticamente as rotas REST para todas as actions do controller, eliminando a necessidade de configuração manual de cada endpoint. Isso segue o princípio de convenção sobre configuração.

Reiniciar o Servidor

Após adicionar o AutoRoute, é obrigatório reiniciar o servidor para que as novas rotas sejam carregadas:

```
./start
```

Aguarde o servidor compilar e confirme que não há erros no terminal antes de testar as URLs.



6. Como Acessar os Relatórios

Com o servidor rodando, os relatórios ficam disponíveis nas seguintes URLs. Substitua `UUID` pelo identificador real da turma desejada.

Relatório	URL de Exemplo
Desempenho de uma turma	<code>/Relatorios/Desempenho?turmaId=UUID</code>
Frequência de uma turma	<code>/Relatorios/Frequencia?turmaId=UUID</code>
Ranking geral	<code>/Relatorios/Ranking</code>

- 📄 Recomenda-se criar botões nas páginas de Turma e Aluno que apontem para esses links, facilitando o acesso direto aos relatórios sem necessidade de digitar a URL manualmente.

7. Conceitos de Programação Funcional Aplicados

A tabela abaixo mapeia cada conceito de Programação Funcional estudado na disciplina com o local exato onde ele aparece no código do SGA-Func.

Conceito	Onde aparece no código
map	Transformar lista de notas em médias individuais por aluno
foldr / fold	Agrupar notas e frequências por aluno usando <code>Map.insertWith</code>
Funções puras	<code>calcularDesempenho</code> , <code>calcularFrequencia</code> , <code>calcularRanking</code>
Imutabilidade	Nenhum dado é alterado; apenas transformações produzem novos valores
Composição	Pipeline de <code>fetch</code> + <code>cálculo</code> + <code>render</code> encadeados no Controller
Lazy evaluation	Listas processadas sob demanda pelo compilador GHC

Programação Funcional na Prática

map — Transformação

Aplica uma função a cada elemento de uma lista, produzindo uma nova lista. No SGA-Func, converte cada grupo de notas em uma tupla (Id Aluno, Double) com a média calculada.

foldr — Agregação

Reduz uma lista a um único valor acumulado. Aqui, agrupa todas as notas e frequências por aluno em um Map, sem modificar os dados originais.

Composição de Funções

O pipeline `fetch → calcular → render` encadeia operações puras, tornando o fluxo de dados previsível, testável e fácil de manter.

Lazy Evaluation

O GHC processa listas sob demanda, evitando computações desnecessárias e melhorando a eficiência do sistema mesmo com grandes volumes de dados acadêmicos.

8. Melhorias Sugeridas

O sistema implementado é funcional e completo para os requisitos da disciplina, mas pode ser estendido com as seguintes melhorias:

1 Exibir nome do aluno

Em vez de exibir o ID, usar `fetchRelated` ou um `join` SQL para mostrar o nome completo do aluno nos relatórios, tornando-os mais legíveis.

2 Filtros avançados

Adicionar filtros por disciplina, período letivo ou professor, permitindo análises mais segmentadas e úteis para a coordenação acadêmica.

3 Formatação de valores

Formatar médias com duas casas decimais e percentuais de frequência com o símbolo %, melhorando a apresentação visual dos dados.

4 Exportação PDF / CSV

Implementar a exportação dos relatórios para PDF ou CSV, facilitando o arquivamento e o compartilhamento com coordenadores e professores.

5 Gráficos de desempenho

Criar visualizações gráficas simples utilizando bibliotecas JavaScript como Chart.js, integradas às views HSX para representar tendências de desempenho.

9. Resumo dos Passos



Criar Controller

Implementar Ações

Funções de Cálculo

Criar Views

Seguindo estes seis passos em ordem, a equipe terá o módulo de relatórios completamente funcional e integrado ao restante do SGA-Func.

10. Conclusão

Com este guia, a equipe consegue implementar relatórios de desempenho completos no SGA-Func, aplicando de forma prática os conceitos de programação funcional — **map**, **reduce/fold**, **funções puras** e **imutabilidade** — exigidos pela disciplina.

Integração Completa

Os relatórios ficam integrados ao restante do sistema — Alunos, Turmas, Disciplinas, Notas, Frequência e Professores — formando um sistema acadêmico coeso e extensível.

Extensibilidade

A arquitetura baseada em funções puras e composição facilita a adição de novos relatórios e funcionalidades conforme as necessidades do projeto evoluem ao longo do semestre.

UNIFOR

Análise e Desenvolvimento de Sistemas

Disciplina

Programação Funcional

Projeto

SGA-Func com Framework IHP